

Bottom-Up Integration of a 6-DOF r6bot in ROS 2 Control and Gazebo

A practical project report based on Modern Robotics, project notes, and the debugging record

Abstract

This report describes our attempt to adapt ROS 2 control demo example 9 from the default rrbot model to an r6bot, a 6-DOF robot arm, with the goal of moving beyond RViz visualization and toward a working Gazebo simulation. The main result is not a new robotics algorithm. The useful result is a clearer engineering workflow for robot integration: understand the physical robot as links and joints, check the generated URDF/xacro model, connect ros2_control with the same joint names, add Gazebo-specific physics and plugin requirements, and only then test the launch behavior. The project reached a real intermediate state: the r6bot could be displayed and controlled in RViz, and the default Gazebo demo could run, but the r6bot Gazebo path still required file-level cleanup and validation. The biggest mistake was treating the adaptation as a rename from rrbot to r6bot. In practice, the rrbot and r6bot files did not expose the same xacro macro structure, and leftover rrbot assumptions created mismatches. This report explains the concepts, debugging steps, evidence, current limits, and next implementation plan without claiming a finished Gazebo validation.

Keywords: ROS 2 control; Gazebo; URDF/xacro; r6bot; forward kinematics; robot description

1. Introduction

The project started with a task that sounded simple: take ros2_control_demos example 9, which is built around rrbot, and make it use r6bot instead. At first, this looked like a naming problem. There were rrbot files, there were r6bot files, and the obvious move was to replace the old robot name with the new one. That was the first trap. A ROS robot demo is not only a group of filenames. It is a stack of linked assumptions: the robot description, xacro macros, joint names, controller interfaces, Gazebo plugin settings, launch arguments, and controller YAML all have to agree with each other.

Once those layers started mixing, the system entered an annoying middle state. Some things looked almost correct. The model could show up, parts of the launch process could run, and a file name might say r6bot. But another file could still be calling an rrbot macro, or a controller YAML file could still be expecting rrbot joint names. That kind of bug is harder than a normal syntax error because the software does not know our intention. It only sees exact names, exact macros, and exact interfaces.

This is why we shifted to a bottom-up approach. In this report, bottom-up does not mean a complicated new method. It means we stopped starting from the final launch file and instead rebuilt the explanation from the robot itself. The order became: links, joints, coordinate frames, robot description, control interface, controller configuration, Gazebo requirements, and finally launch behavior. That order matched the actual debugging better than jumping from one error message to another.

The robotics reference for the conceptual side is Modern Robotics by Lynch and Park. We use it mostly for vocabulary and structure, not to make the project look more theoretical than it is. The useful ideas are basic but important: a robot arm is a mechanism made from links and joints; a configuration describes the robot state; forward kinematics connects joint variables to the end-effector pose; and coordinate frames tell us how each body is positioned relative to another. These ideas map directly to URDF. A URDF file is basically a software version of the physical robot structure.

The project did make progress, but it did not end as a fully validated Gazebo simulation. The ROS 2 environment issue was solved by sourcing the correct setup file. The build loop was improved by building only the needed package instead of the whole workspace. The default example 9 Gazebo launch worked. The r6bot model could appear and be controlled in RViz. The remaining work is the stricter part: making r6bot behave correctly inside Gazebo, where visual geometry is not enough. Gazebo also needs collision geometry, inertia,

mass, a plugin, `ros2_control` hardware interfaces, and controller YAML that matches the actual generated robot description.

So the point of this report is not to dress up an unfinished simulation as a finished result. The point is to show what was actually learned from the integration attempt, which assumptions failed, and what exact checks should come next. For a robotics software project, that is still a useful outcome because the difference between a displayed robot and a simulated robot is not cosmetic. It is the difference between a picture of a mechanism and a mechanism that the software can treat as a physical system.

2. Conceptual Background: From Robot Arm to Robot Description

Modern Robotics describes a robot mechanism as rigid bodies connected by joints, with actuators moving the joints and an end-effector attached to one part of the chain. That sentence sounds almost too simple, but it is exactly what we needed to remember while editing URDF and xacro files. A robot description is not just a mesh or a 3D drawing. It is a structured list of bodies, connections, transforms, and sometimes physical properties. If the structure is incomplete or inconsistent, the robot may still look acceptable in a viewer but fail when control or physics is added.

2

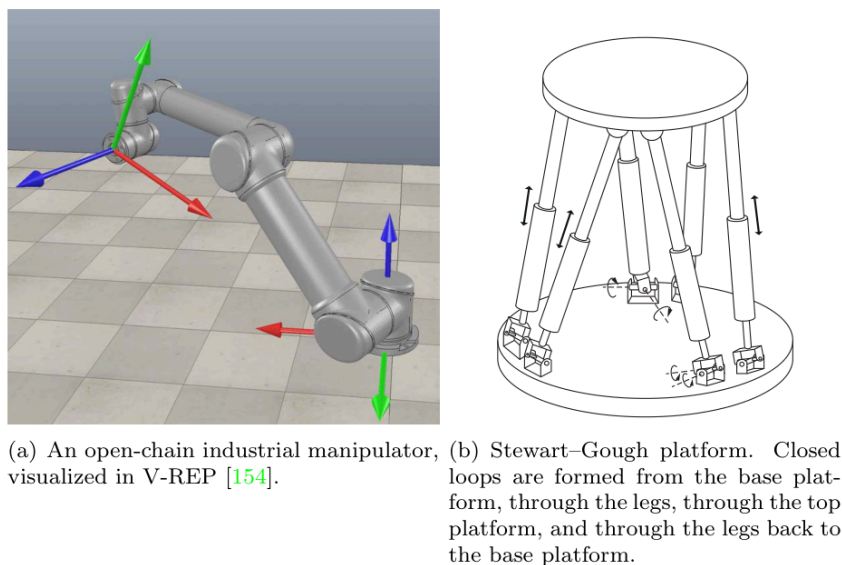


Figure 1. Open-chain and closed-chain mechanisms used as background for the r6bot description problem. Source: Lynch and Park (2017).

The first concept that mattered was configuration. For a robot arm, the configuration is usually represented by joint variables. In our notes, forward kinematics was written in a direct way: use joint angles to calculate the 3D end-effector position relative to the base. That is not a full formal derivation, but it captures the practical direction. The robot has angles q_1 , q_2 , q_3 , and so on. The task cares about the end-effector pose. Forward kinematics moves from the joint variables to the pose, while inverse kinematics would try to move from a desired pose back to joint values.

The second concept was coordinate frames. Every joint origin and every joint axis in a robot description depends on frames. If the frame is wrong, a joint can rotate about the wrong axis, a link can appear in the wrong place, or the end-effector can be offset even though the visual geometry looks close. Modern Robotics uses rotation matrices and homogeneous transformations to represent rigid-body motion. Even the basic

two-dimensional rotation matrix was helpful because it reminded us that the same physical point can have different coordinates when the frame rotates:

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

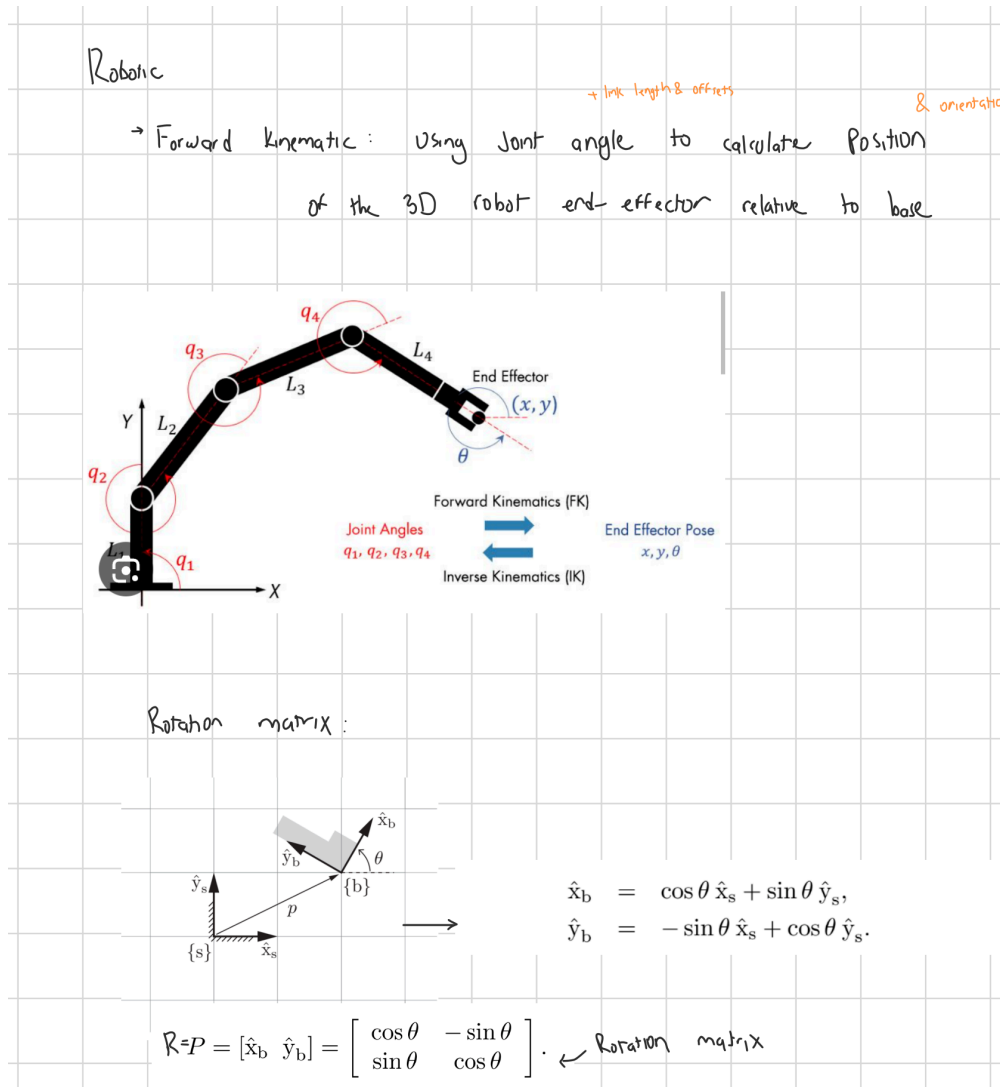


Figure 2. Project notes connecting joint angles, forward kinematics, end-effector pose, and rotation matrices. Source: project robotics notes.

This matters in ROS because a joint origin tag is not decoration. It tells the software the transform between parent and child link frames. If that transform is wrong, the robot is wrong in a structural way, not just in a visual way. The same issue appears when we attach the robot to a world link. If the fixed joint points to a child link name that is not actually the root link of r6bot, the model is being attached to the wrong thing.

The third concept was the discipline of assigning frames before calculating or coding. Modern Robotics often uses product-of-exponentials tools, while many class notes use Denavit-Hartenberg parameters. We did not need to finish a full DH model for the final ROS file, but the DH habit was useful. It says: do not guess coordinates from a picture. Assign frames, identify joint axes, measure offsets, and then compose transforms. That habit translated directly into the URDF work. We should not guess a macro call from r6bot just because the file shape looks similar. We should check what r6bot actually defines, then build upward from that.

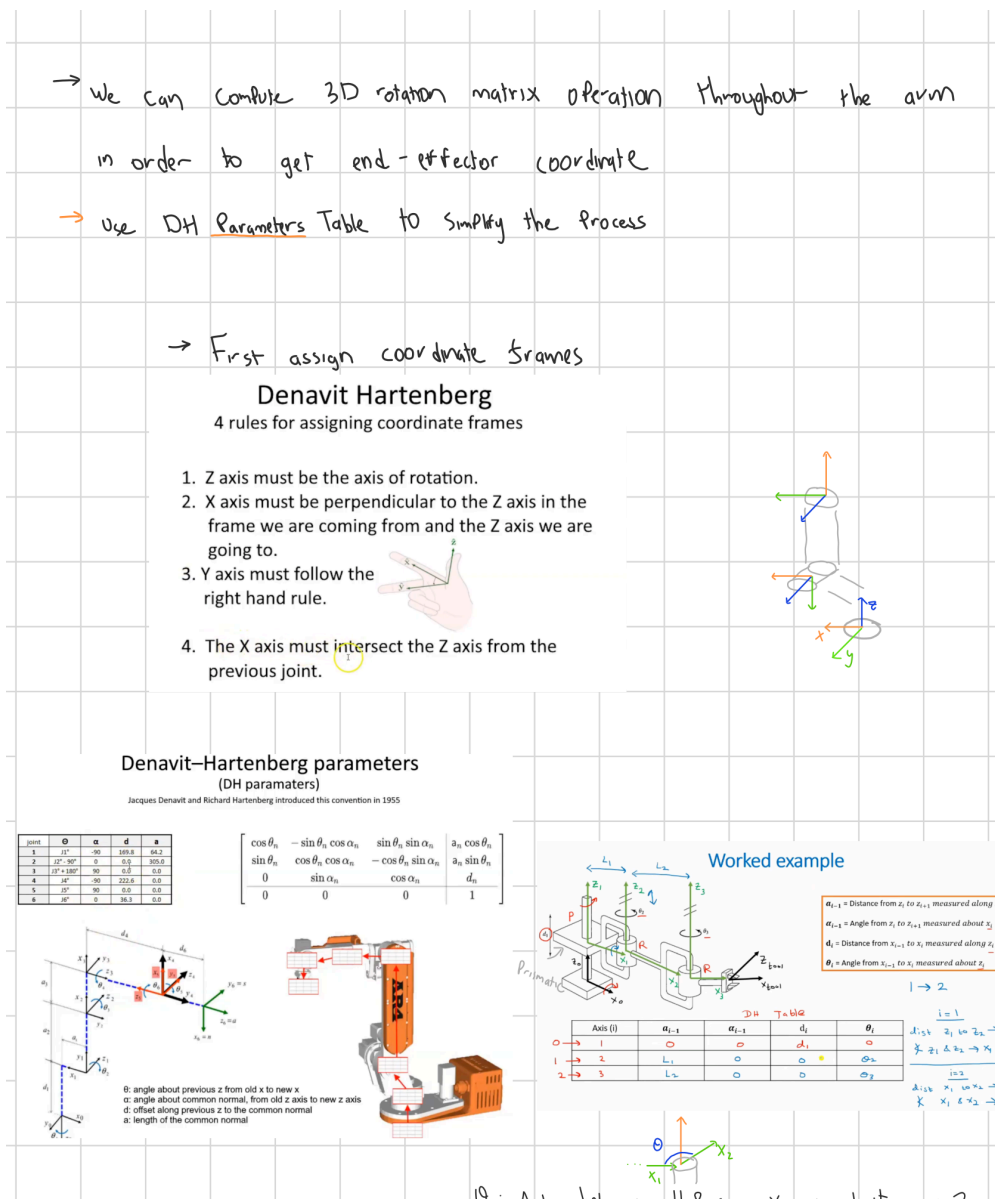


Figure 3. Project notes on Denavit-Hartenberg frame assignment and parameter-table thinking. Source: project robotics notes.

The r6bot link-labeling sketch was also useful for this reason. It forced us to separate the physical robot idea from the copied r6bot assumptions. Before talking about ros2_control or Gazebo plugins, we needed to ask what bodies and joints the r6bot model was supposed to contain. The exact names still have to come from the generated URDF, but the physical structure should be clear first: base side, link sections, wrist/end-effector side, and actuated joints between them.

A final distinction is task space versus configuration space. In a robot arm, configuration space is usually described by joint variables. Task space is described by what the end-effector should do, such as position and orientation. This project was not yet doing trajectory planning or grasping, but the distinction still helped. RViz can show a configuration changing visually. Gazebo must simulate bodies with collision and inertia. The same joint motion that looks fine in RViz may fail in Gazebo because the simulator needs a more complete robot description.

This conceptual background is not separate from the debugging. It explains why the first approach broke down. We tried to adapt a robot by renaming a working example. But a robot description is a physical and kinematic structure, not a label. The better question became: does every layer describe the same robot with the same links,

joints, roots, control interfaces, and simulator assumptions? Once we used that question, the debugging became less random.

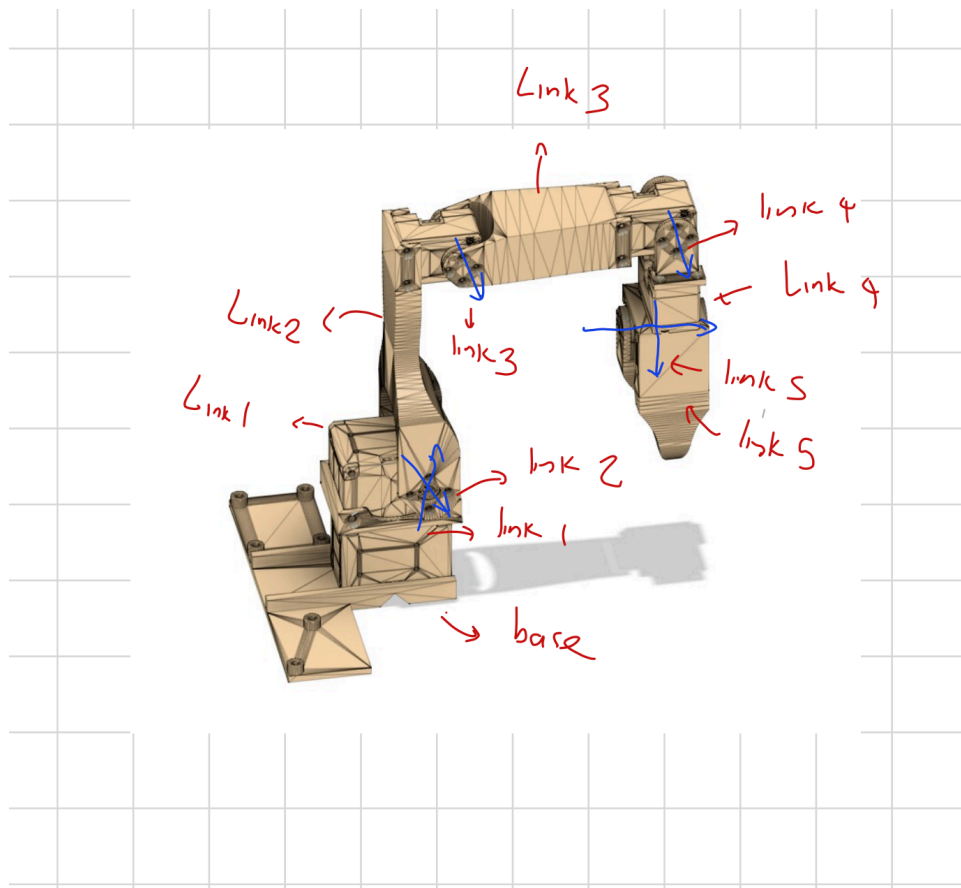


Figure 4. r6bot link-labeling sketch used to keep the physical structure separate from copied rrobot assumptions. Source: project robotics notes.

3. Project Goal, Scope, and Evidence Sources

The practical goal was to modify `ros2_control_demos` example 9 from `rrobot` to `r6bot`. The target was Gazebo integration, not only RViz display. This difference is important because RViz and Gazebo answer different questions. RViz asks whether the robot model can be displayed and animated from joint states. Gazebo asks whether the robot can exist as a simulated physical system with bodies, collisions, inertial values, control interfaces, and plugin configuration.

The implementation evidence comes mainly from the debugging record. It shows several progress points that are worth keeping: the ROS 2 Jazzy setup had to be sourced, the workspace setup had to be sourced after build, the build command was narrowed to the needed package, the default example 9 Gazebo launch could run, and the `r6bot` model could appear and be controlled in RViz. These are not final Gazebo validation, but they are real progress. They also narrow the problem. If the default Gazebo demo works, then the machine and basic Gazebo setup are probably not the main issue. The remaining issue is more likely inside the `r6bot` adaptation.

The conceptual evidence comes from the project notes. The notes include forward kinematics, rotation matrices, three-dimensional rotations, Denavit-Hartenberg frame assignment, parameter-table thinking, and the `r6bot` link sketch. They are rough notes, but that is part of why they are useful. They show the learning path behind the code edits. We were not only copying XML. We were trying to understand what the XML was supposed to represent.

The formal reference is Modern Robotics by Lynch and Park. The book gives the clean terminology for rigid bodies, joints, configuration space, task space, workspace, rotation matrices, transformations, and control. The report uses that reference to keep the robotics language correct. When we say configuration, we mean the robot state in the robotics sense, not only a software configuration file. When we say end-effector pose, we mean the physical pose of the tool frame relative to a base or world frame.

The scope is also limited on purpose. This report does not include a finished dynamics benchmark, controller performance test, or measured validation of inertial parameters. It focuses on the integration path and the file-level issues that blocked the move from RViz to Gazebo. That boundary matters because a project report should say where the work actually reached. The current state is: RViz-level display/control works, the default Gazebo baseline works, and the r6bot Gazebo integration path is understood but not fully validated.

Table 1. Difference between RViz-level success and Gazebo-level success.

Layer	Minimum needed	Meaning for this project
RViz	Visual geometry and joints	The robot can appear and move visually, so the description is not completely broken.
Gazebo	Visual geometry, collision geometry, mass, inertia, Gazebo plugin, ros2_control interfaces, and controller YAML	The robot must behave as a simulated physical and controllable system, not only as a displayed model.
Main risk	Assuming RViz success equals Gazebo success	This hides the remaining physics and controller work and makes the project look more complete than it is.

4. Method: Bottom-Up Debugging Workflow

The method was to rebuild the system layer by layer. The first layer was the ROS environment. This is not the exciting part of robotics, but it matters. A workspace can fail before any robot-specific problem appears. In our case, the ROS 2 setup file had to be sourced first:

```
source /opt/ros/jazzy/setup.bash
```

After building, the workspace install setup also had to be sourced. The build loop was made smaller by building only the relevant demo package instead of rebuilding the whole repository every time:

```
cd ~/ws2
colcon build --packages-up-to ros2_control_demo_example_9
source install/setup.bash
```

That change sounds small, but it affected the way we debugged. If every edit requires a long rebuild, we start avoiding careful tests. A shorter loop made it easier to change one thing, rebuild, and check whether the change actually mattered.

The second layer was the baseline demo. Before blaming r6bot, we needed to know whether the original example could run. The debugging record says the default example 9 Gazebo launch worked with rrobot. That gave us a baseline. It meant the ROS installation, package paths, and basic Gazebo launch were not completely broken. From that point, the remaining errors could be treated as r6bot adaptation problems instead of system-wide problems.

The third layer was the robot description. This is where the main mistake became clear. The r6bot.urdf.xacro file contained useful r6bot parts, including r6bot_description and r6bot_ros2_control ideas, but it also carried leftover rrobot logic. The important mismatch was not just the word rrobot. The macro interfaces were different. The rrobot macro supports a call pattern with parameters such as parent, prefix, and origin. The r6bot_description macro in this project only exposes the r6bot_description call. That means copying the rrobot macro call and changing the name does not work.

For rrobot, a call like this makes sense because the macro was written to accept those parameters:

```
<xacro:rrbot parent="world" prefix="$(arg prefix)">
```

For the r6bot description we had, the safer direction was to include the real r6bot description file and call the macro that actually exists:

```
<xacro:include filename="$(find ros2_control_demo_description)/r6bot/urdf/r6bot_description.urdf.xacro"/>  
<xacro:r6bot_description/>
```

This was the clearest example of why the bottom-up approach was useful. A rename approach asks, 'How do we replace rrbot with r6bot?' A bottom-up approach asks, 'What does the r6bot description actually provide?' The second question changes the implementation.

The fourth layer was the world attachment and root link. Many demos attach a robot to a world link through a fixed joint. That is reasonable, but the child link name has to match the real root link of the robot. If the r6bot root is not base_link, then a world_to_base fixed joint using base_link is wrong even if the name looks normal. This kind of mistake is easy to miss because base_link is a common name, but common is not the same as correct for this file.

The fifth layer was ros2_control. A visual robot description can exist without being controllable. ros2_control needs command and state interfaces for specific joints. For r6bot, this means the ros2_control xacro file must use the exact joint names from the generated robot description. If the controller YAML lists joint_1 but the URDF contains shoulder_pan_joint, the controller is not connected to the robot. The software does exact string matching; it does not infer that two names refer to the same physical joint.

The sixth layer was Gazebo. Gazebo is not another version of RViz. It needs enough information to simulate physical bodies. That includes collision geometry, inertial values, mass, plugin settings, and ros2_control hardware interfaces. A Gazebo reference block that only changes material color is not enough. It may improve appearance, but it does not add mass or controller interfaces.

The seventh layer was validation. Instead of using one big launch as the only test, the better path is a sequence: generate the URDF from xacro, inspect links and joints, check the root link, compare joint names with ros2_control, compare the controller YAML, check for inertial and collision tags, confirm the Gazebo macro name, and then launch Gazebo. This is slower than pressing launch first, but when something fails it gives a much clearer clue about where the mismatch is.

5. Results and Debugging Findings

The first result was a cleaner environment and build loop. The project found that sourcing the ROS 2 Jazzy setup and the workspace setup were necessary. It also found that building only ros2_control_demo_example_9 was enough for this task. This is not a glamorous robotics result, but it is part of making the work repeatable. When the build loop is unreliable, every later debugging step becomes less trustworthy.

The second result was that the original Gazebo example could launch. This matters because it ruled out some large categories of failure. If the original rrbot example had failed too, the problem might have been Gazebo installation, package paths, or ROS setup. Since the original example worked, the r6bot failures were more likely caused by the adaptation itself.

The third result was that r6bot could appear and be controlled in RViz. This showed that the robot description was not completely broken. The model existed, the joint-state path worked enough for visualization, and we had a useful visual checkpoint. However, this was also where the project could easily be overread. RViz success does not prove Gazebo success. It only proves that the display and joint-state side reached a working level.

The fourth result was identifying the main structural bug: leftover rrbot assumptions inside r6bot files. The wrong pieces included rrbot macro calls, rrbot ros2_control calls, and rrbot Gazebo calls. More importantly, the rrbot and r6bot macro structures were not the same. rrbot could accept a parent/prefix/origin style call, while r6bot_description had a simpler macro call. This made a direct rename invalid.

The fifth result was a clearer file responsibility map. Before this project, it was tempting to treat all xacro files as one general robot description. The debugging showed that each file has a different job. The top-level `r6bot.urdf.xacro` assembles the robot. The `r6bot_description` file defines physical links and joints. The `r6bot.ros2_control` file exposes command and state interfaces. The `r6bot.gazebo.xacro` file adds Gazebo-specific plugin and settings. The `r6bot_controllers.yaml` file configures the controllers. The launch file ties those pieces together.

Table 2. Bottom-up file responsibility map for the r6bot adaptation.

File or component	Responsibility	Current check
<code>r6bot.urdf.xacro</code>	Top-level robot assembly	Remove leftover rrobot macro calls and include only r6bot-specific pieces.
<code>r6bot_description.urdf.xacro</code>	Physical links and joints	Confirm root link, link names, joint names, collision tags, and inertial tags.
<code>r6bot.ros2_control.xacro</code>	Control interfaces	Use exact r6bot joint names and correct command/state interfaces.
<code>r6bot.gazebo.xacro</code>	Gazebo plugin/settings	Define <code>r6bot_gazebo</code> and point the plugin to <code>r6bot_controllers.yaml</code> .
<code>r6bot_controllers.yaml</code>	Controller manager configuration	Joint names must match the generated URDF and ros2_control interfaces.
Launch file	Start robot, Gazebo, and controllers	Load <code>urdf/r6bot.urdf.xacro</code> with <code>use_gazebo:=true</code> ; rename later for clarity if needed.

The remaining likely problems also became more specific. The controller YAML may not match the generated URDF joint names. The `ros2_control` xacro may expose interfaces for joints that do not exist or may miss joints that should be controllable. The fixed world joint may point to an assumed `base_link` instead of the real r6bot root. Some links may still be missing inertial or collision tags required for Gazebo. The Gazebo macro may still use an old `rrbot` name or may point to `rrbot_controllers.yaml`. These are concrete checks, not vague guesses.

Another useful finding was that Gazebo reference blocks should not be mistaken for physics setup. A block that sets material color can change how a link looks in Gazebo, but it does not create inertia, collision, mass, or control. This distinction prevented a false fix. It would be easy to keep adding appearance settings and think the Gazebo problem is being solved, when the actual issue is still a missing physical or control interface.

The current result can be stated simply: the project reached working RViz visualization/control for r6bot, confirmed that the default Gazebo demo can run, and produced a clear checklist for r6bot Gazebo integration. It has not yet proven a complete r6bot Gazebo simulation. That boundary is important because the next work should be validation, not storytelling.

6. Discussion: What the Project Actually Taught

The project taught us that robotics integration has many almost-correct states. A robot can be almost correct visually, almost correct in xacro, almost correct in controller YAML, and still fail as a full system. This is different from a small programming exercise where a syntax error points to one line. In ROS 2 and Gazebo, the visible error may appear during launch even though the real mistake is a wrong assumption in the robot description several files earlier.

One lesson is that names are real interfaces. In human language, `joint_1` and `shoulder_joint` might describe the same part. In ROS, they are different strings unless we explicitly make them the same. The URDF joint names, `ros2_control` joint names, and controller YAML joint names must all refer to the same objects. If they do not, the controller is trying to command a robot that does not exist in the generated description.

Another lesson is that RViz is useful but limited. It gave fast feedback and helped confirm that the r6bot model was not completely broken. Still, it is a visualization tool. It does not force the same level of physical completeness that Gazebo does. Gazebo cares whether a link has collision geometry, whether inertia is present,

whether a plugin is loaded, and whether `ros2_control` can expose hardware interfaces. A robot that only looks right is not yet a simulated robot.

The formal robotics concepts helped most when they stayed close to the actual problem. Writing many equations would not have made this report better. The useful ideas were links, joints, frames, transformations, configuration, and forward kinematics. Those ideas explained why parent-child relationships matter in URDF, why joint origins are important, and why an arm should be checked as a chain of bodies instead of as a collection of shapes.

The Denavit-Hartenberg notes helped in the same practical way. A DH table is basically a reminder to assign frames before calculating. The ROS version of that habit is to assign file responsibilities before editing. The mistake was not one random XML line. The mistake was mixing `rrbot` file logic with `r6bot` model logic. DH thinking says not to mix coordinate frames casually; this project taught the same lesson for robot macros and controller names.

The bottom-up workflow also reduced panic. When Gazebo fails, it is tempting to search the last error message and paste the first fix that looks related. Sometimes that works, but it does not build much understanding. A bottom-up checklist is calmer: first check whether `xacro` generates a valid robot, then inspect links and joints, then compare control interfaces, then compare controller YAML, then check Gazebo physics and plugin settings, and only after that treat the launch file as the final integration step.

There are limitations in this report. It is based on the project files, notes, and debugging record, not a final Gazebo log showing a fully simulated `r6bot` moving under controller command. It does not verify the robot dynamics, it does not prove the inertial values are physically accurate, and it does not evaluate controller performance. It also does not mathematically validate the `r6bot` kinematics through a completed DH table or product-of-exponentials model. Those would be next-stage validation tasks, not current results.

A practical next step is to turn each suspected mismatch into a test. For joint names, generate the URDF from `xacro` and list all joints, then compare that list to `r6bot.ros2_control.xacro` and `r6bot_controllers.yaml`. For the root link, inspect the generated tree and confirm that the fixed world joint attaches to the real root. For Gazebo readiness, check every link for collision and inertial tags. For plugin settings, confirm that `r6bot.gazebo.xacro` loads `r6bot_controllers.yaml`. For launch clarity, rename files only after the content is correct, so the names do not hide old assumptions.

The broader habit is useful beyond this single robot. When adapting a demo, we should not start by asking which words to replace. We should ask what assumptions the demo made. The `rrbot` demo assumes an `rrbot` model, `rrbot` macros, `rrbot` joint names, `rrbot` controllers, and `rrbot` Gazebo settings. `r6bot` needs equivalent pieces, not just renamed pieces. Copying the shape of a solution can be useful. Copying hidden assumptions is usually where the bug starts.

7. Implementation Plan for the Next Working Version

The next working version should begin with `r6bot.urdf.xacro`. That file should be cleaned until it contains only `r6bot` logic. It should include `r6bot_description.urdf.xacro`, call `xacro:r6bot_description`, attach the actual base/root link to world if needed, include `r6bot.ros2_control.xacro`, call `xacro:r6bot_ros2_control`, and include `r6bot.gazebo.xacro` only when `use_gazebo` is true. Any remaining `xacro:rrbot`, `xacro:rrbot_ros2_control`, or `xacro:rrbot_gazebo` call should be removed.

After that, the generated URDF should be inspected directly. This step should not be skipped. `Xacro` can hide the final result because macros expand into a generated robot description. The generated URDF should be checked for link names, joint names, parent-child relationships, root link, inertial tags, collision tags, and `ros2_control` tags. If this generated file is wrong, Gazebo debugging should wait. A simulator cannot fix a robot description that already describes the wrong structure.

The controller YAML should then be rewritten from the generated joint list, not from memory. This avoids a common mistake: using names that look reasonable but do not match the actual model. For a basic position-control demo, each controllable r6bot joint should expose the expected command interface and state interfaces. The YAML should not include joints that are visual-only, fixed, missing, or named differently in the URDF.

The Gazebo xacro file should be checked next. It should define a macro named `r6bot_gazebo`, not `rrbot_gazebo`, and it should point the plugin parameters to `config/r6bot_controllers.yaml`. Any reference to `rrbot_controllers.yaml` is a sign that the adaptation is incomplete. Appearance settings can stay if useful, but they should be treated as appearance only. The physics and control requirements must be checked separately.

Finally, the launch file should be cleaned. Temporarily, it can keep an old filename if that is what the package expects, but the content should load `urdf/r6bot.urdf.xacro` with `use_gazebo:=true`. Later, renaming `rrbot_gazebo.launch.py` to `r6bot_gazebo.launch.py` would make the package easier to read. File names are not the main technical issue, but clear names reduce future mistakes.

A compact validation order would be: generate URDF, list links and joints, compare controller names, check collisions and inertia, launch robot_state_publisher, check RViz, launch Gazebo, then load controllers. If any step fails, fix that layer before moving upward. This keeps the debugging from turning back into random trial and error.

The definition of done for the next version should also be written down before testing. For this project, done should mean more than seeing the model on the screen. A stronger definition would be: xacro generates without errors, the generated URDF contains the expected r6bot links and joints, every controllable joint appears in `ros2_control` and controller YAML with the same spelling, Gazebo loads the model without missing-inertia or missing-plugin errors, the controller manager starts, and a simple position command causes the expected joint motion in simulation. Recording these checks would make the final claim easier to defend because each success would correspond to a specific layer of the stack, not just a screenshot that looks correct.

We should also keep a small run log beside the files. Each test should record the command, the file version, the error message if it failed, and the next change made. That is not extra paperwork for its own sake. It prevents us from changing three files at once and then not knowing which edit actually fixed or broke the launch.

8. Conclusion

This project was about adapting a ROS 2 control demo from `rrbot` to `r6bot` and moving from RViz-level success toward Gazebo-level simulation. The most important result is not a perfect final demo. The most important result is knowing why the system was not finished yet and which layers must be made consistent before Gazebo can be trusted.

The bottom-up approach was the right fit for this problem. Starting from links, joints, frames, and robot description made the later ROS files easier to understand. Modern Robotics gave the formal language for configuration, forward kinematics, task space, and mechanisms. The project notes connected that theory to the way we were actually learning: rotation matrices, DH parameters, frame assignment, and link labeling. The debugging record connected the theory to implementation: which commands worked, which files mattered, and which mismatches remained.

The final state is clear. We have not proven a fully working r6bot Gazebo simulation yet. We have moved from a loose rename attempt to a structured integration plan. The next milestone is to make the generated r6bot URDF, `ros2_control` joint interfaces, Gazebo plugin, and controller YAML all describe the same robot with the same names. Once that is true, Gazebo debugging becomes a normal validation problem instead of a guessing game.

References

Lynch, K. M., & Park, F. C. (2017). Modern Robotics: Mechanics, Planning, and Control. Cambridge University Press.

Project debugging summary. (n.d.). Summary of r6bot / ROS 2 control demo example 9 debugging record. Unpublished project document.

Project robotics notes. (n.d.). Forward kinematics, rotation matrix, Denavit-Hartenberg parameters, and r6bot link-labeling notes. Unpublished project notes.

Appendix A. Practical Validation Checklist

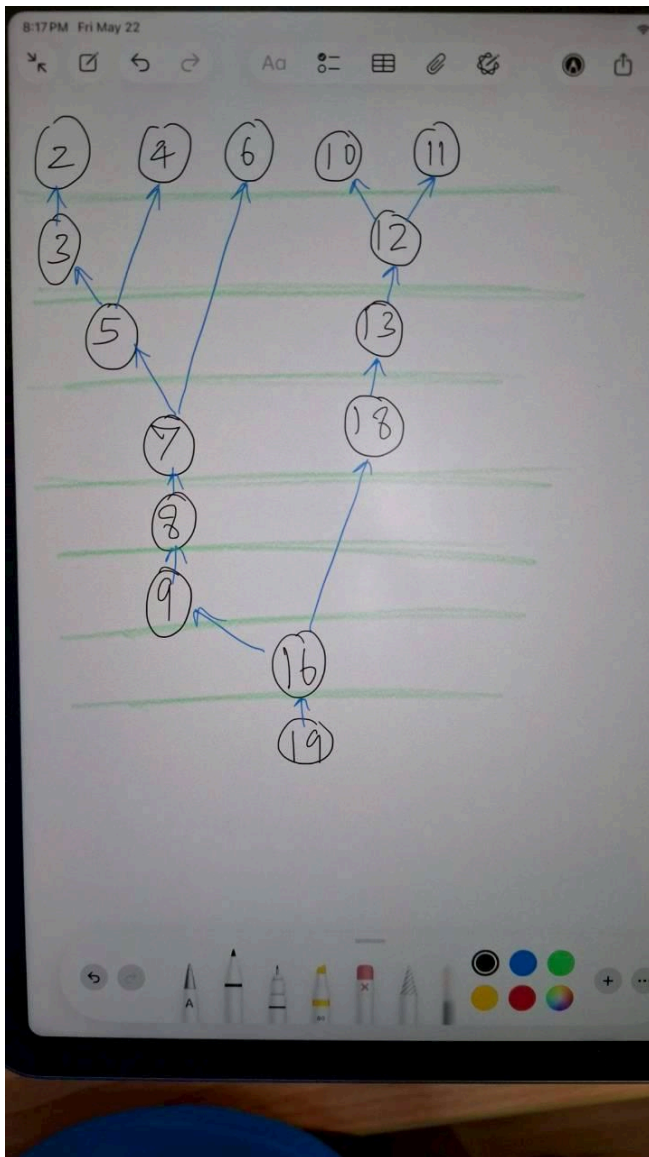
- Generate the r6bot URDF from xacro and inspect the final link and joint names.
- Confirm the fixed world joint uses the real r6bot root link, not an assumed base_link.
- Check that every Gazebo link has collision and inertial tags.
- Compare URDF joint names with r6bot.ros2_control.xacro and r6bot_controllers.yaml.
- Confirm r6bot.gazebo.xacro defines r6bot_gazebo and loads r6bot_controllers.yaml.
- Launch Gazebo only after the generated robot description is internally consistent.

Reflection :

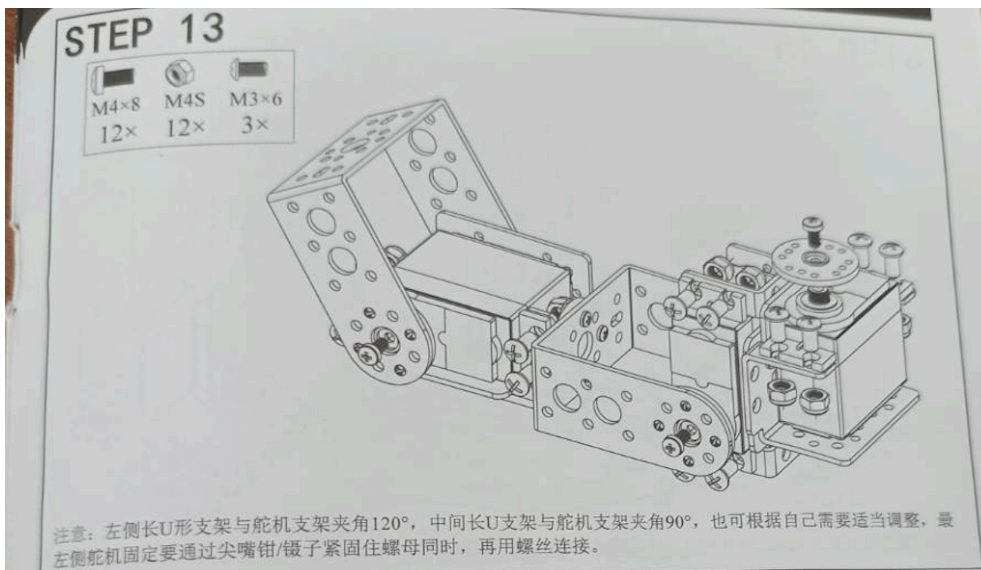
蔡定宇:

Responsibility: Assembly and ROS2 Compatibility Support

During this robotic arm project, my main responsibility was assembly. I worked mainly on steps 4, 5, 11, 18, and 16. The job distribution pipeline was given by Jaijai, so each member already had a section to handle. My assigned steps went smoothly at first. Step 4 was simple because it mainly involved installing the servo and angle plate, choosing the correct screws, and following the drawing. Step 5 continued from the same base structure, so it was manageable. Step 11 had more brackets and screws, but once I understood how the pieces connected, the process became smoother.



The main issue happened when I tried to help with step 13. The manual said that the left long U-shaped bracket and servo bracket should form around 120 degrees, while the middle long U-shaped bracket and servo bracket should form around 90 degrees. It also said the angle could be adjusted depending on the need. Looking back, this was probably only an adjustment note. At that moment, I connected it to motor alignment and started thinking the motor direction might be wrong.



My mistake was not noticing a possible issue. The mistake was not proving it before acting on it. I focused too much on one note and did not compare it enough with the full assembly flow from steps 13 to 18. I should have checked the final posture, tested the servo's neutral position and rotation range, and confirmed the horn direction before changing the motor orientation. Instead, I moved from suspicion to action too quickly.

This affected the team because the assembly was a shared pipeline. Once the motor direction was changed incorrectly, the later steps became harder to continue because the arm no longer matched the expected orientation. It caused extra disassembly and debugging. This showed me that even a small mechanical change can slow down the whole team if the decision is not verified clearly.

The teamwork lesson is that I need to communicate my uncertainty in a more testable way. I did not make the decision silently, but I should have asked the team to verify the evidence with me more clearly, even if the evidence ended up opposing my first thought. A better approach would have been to say, "I think the motor direction might be wrong, but let's test the servo angle and compare it with the manual before changing it." That would have turned my concern into a shared verification step instead of letting my interpretation lead the decision.

Besides assembly, I also observed the ROS2 compatibility debugging for the r6bot. I was not the main person writing the simulation code, but I learned that adapting the demo was not just a matter of renaming rrobot to r6bot. Each part of the system had to match: the robot description, joint names, controller file, Gazebo settings, and launch file. If one file still followed the old rrobot setup, the simulation could fail. This reminded me of the same lesson from assembly: before changing something, we need to check what the whole system depends on.

That software part connected directly to my assembly mistake. In assembly, I changed a physical part before fully verifying the mechanical behavior. In ROS2 integration, the same mistake would be changing files before checking what each file actually controls. RViz working also does not automatically prove that Gazebo is working, because Gazebo needs collision, inertia, mass, plugin

settings, and correct controller interfaces. Both the mechanical and software sides showed the same principle: a result is not reliable just because it looks reasonable.

This is also probably why our major puts so much emphasis on proving. Proof is not only for math exams. It is a way of thinking before we act. When we make a claim, change a circuit, modify code, reverse a motor, or debug a system, we should ask what evidence supports the decision. In this project, my assumption was not completely random, but it was not proven enough. Engineering needs that proof-like discipline because one unsupported assumption can create real errors in the system.

Overall, my contribution was mainly on the physical assembly side, but the project taught me a broader engineering habit: verify before changing. My step 13 mistake showed me that overthinking is not the same as being careful. Real carefulness means checking evidence, testing assumptions, and being willing to let the result prove my first idea wrong. Next time, before reversing a motor, changing a structure, or touching a shared part of the pipeline, I should first make the verification process clear with the team. That is the difference between pushing an idea and doing careful engineering.

Jai Jai :

Responsibility: Procurement, Material Planning, and Team Coordination

During this robotic arm project, one of my main responsibilities was handling procurement and material preparation. At first, I thought buying parts would just mean finding the components and placing orders. After actually doing it, I realized procurement is a serious part of engineering. It is not only about buying things. It is about understanding what the team needs, checking whether the parts are suitable, comparing sources, managing cost, and making sure the materials arrive in time for the project to continue.

For the higher-end components, such as servo motors and sensors, I had to look through distributor channels like Mouser, Digi-Key, element14, and MISUMI. This was my first time independently trying to access professional or mass-produced components instead of only using cheap hobby parts. That experience made me understand that real engineering projects depend on reliable supply chains. The same component can have different specifications, prices, availability, and delivery times depending on the supplier. If I chose the wrong part or ordered too late, the whole project could be delayed. Because of that, I had to be more careful and think beyond just whether the part looked correct.

Besides ordering online, I also helped buy materials from nearby electronics and hardware shops for prototyping and quick fixes. These included bolts, nuts, screws, wires, and heat shrinks. These small items may not look as important as motors or sensors, but during assembly, they became very necessary. Sometimes the team needed a quick replacement or a small adjustment, and waiting for online delivery would not be practical. This taught me that engineering work needs both long-term planning and short-term improvisation.

Another important part of procurement was funding. Since the project required money before we could start building, we had to gather as many members as possible and raise funds together. This was not only a financial issue. It also required building alignment and trust within the team. Everyone had to believe that the money would be used properly and that the materials were being bought for the purpose of learning and collaboration. I had to be responsible because the materials were not only for myself; they were shared resources for the whole group.

Through this project, I learned that procurement is connected to leadership. A good procurement decision supports the technical side before assembly even begins. If the parts are wrong, missing, late, or unsuitable, even a good design cannot move forward smoothly. I also learned that communication is important because the person buying the materials must understand what the builders, coders, and testers actually need.

Overall, this project taught me that engineering is not only about design and assembly. It also depends on planning, responsibility, trust, and resource management. Procurement may happen before the visible work begins, but it strongly affects whether the team can succeed later.

Frans :

For this project, I am mainly in charge of the simulation part using ROS2. The goal of my task is to become familiar with the robot simulation process first, and then try to integrate it with the real 6DOF robotic arm that our group is working on. I will write my personal reflection with the following format:

Outline

Scope

My approach to the task

Difficulty

What I got right

Reflection as a whole

As an outline, I first needed to understand the basic concept behind the project. ROS2 is basically the framework that allows different parts of the robot system to communicate with each other. It is not only about writing one simple program, but also about connecting many parts together, such as the robot description, the controller, the simulation environment, and the launch files. In simple words, ROS2 helps define how the robot should move, how the joints are controlled, and how the system communicates between different nodes.

For the simulation side, there are two important tools that I had to understand: RViz and Gazebo. At first, I thought both of them were just simulation software, but later I realized that they are used for different purposes. RViz is mainly used for visualization. It allows us to see the robot model, check the joints, frames, and overall robot structure. However, RViz does not really simulate physics. Gazebo is different because it includes the physics environment. This means Gazebo can simulate things like mass, inertia, collision, gravity, and how the robot behaves when it is controlled. This difference was quite important because getting the robot to appear in RViz does not mean the robot is ready to work in Gazebo.

The scope of my task was fairly simple in the beginning. I wanted to provide a reliable simulation medium so that our group could roughly test whether the robotic arm system can work before directly dealing with the physical robot. Since the actual robot arm involves real hardware, motors, joints, and mechanical structure, having a simulation first is helpful. It gives us a safer environment to test ideas and understand the control flow before moving to the real system. My main focus was not to create a perfect robot simulation from zero, but to understand how to use an existing ROS2 control demo and modify it so it can fit better with the 6DOF robot arm that our group is using.

My approach to this task was to first look for useful references online, especially from GitHub. I tried to find a repository that already had ROS2 control examples, because starting everything from scratch would be too difficult and not realistic for the time we had. I found the `'ros2_control_demos'` repository and started by testing the existing examples. At first, I tried to run the examples directly, but I quickly faced problems with the ROS2 environment and package building. For example, I had to source the ROS2 setup file first, then build the workspace using `'colcon'`. I also learned that building the whole repository was not always the best idea because there are many examples, and if some dependencies are missing, many packages can fail at the same time. So I narrowed the task down and focused mainly on example 9.

One of the biggest difficulties was navigating through other people's codebase. The repository was not written by us, so I had to slowly understand the structure. There were many folders such as launch files, URDF files, Xacro files, controller YAML files, Gazebo files, and description packages. At first, it was confusing because I did not know which file was responsible for what. For example, I thought that changing the robot model in one file would be enough, but later I realized that the launch file, the robot description, the controller configuration, and the Gazebo plugin all need to match each other.

Another difficulty was understanding the difference between `'rrbot'` and `'r6bot'`. The original example 9 used `'rrbot'`, while I wanted to use the 6DOF robot arm model from another directory. I initially

thought I could just rename some parts from ``rrbot`` to ``r6bot``, but that did not work properly. Later I understood that Xacro macros are not just names. They have their own parameters and structure. The ``rrbot`` macro had parameters like parent, prefix, and origin, but the ``r6bot_description`` macro did not have the same structure. This caused confusion because I was trying to apply the structure of one robot to another robot that was built differently.

I also struggled with the Gazebo part. Getting the robot shown in RViz was already a good step, but Gazebo required more things. Gazebo needs the robot to have proper physical properties such as mass, inertia, and collision. It also needs a Gazebo ROS2 control plugin so that the simulated robot can connect to the controller system. This made me realize that simulation is not just about showing a 3D model. The robot has to be described in a way that the physics engine can understand. Otherwise, even if the robot appears, it may not move correctly or the controllers may not connect.

What I got right was that I managed to get the simulation process partially working and I understood the general workflow much better than before. I was able to run the ROS2 control demo, open the robot in RViz, and control it. I also managed to modify example 9 so that it could use the `r6bot` model instead of only staying with the default `rrbot`. Even though the Gazebo part still required more work, I think this was already a meaningful progress because I understood what was missing instead of just randomly changing files.

As a whole, I learned that robotics software is not only about coding one file. It is more about connecting many layers together. There is the robot description, the joint names, the controller interfaces, the launch file, the simulation software, and the actual command topic. If one part does not match, the whole system can fail. This was frustrating at times because the error messages were not always easy to understand. Sometimes the problem was just a wrong file path, sometimes it was a missing package, and sometimes it was because I copied the structure from another robot without fully understanding it.

I also learned how to look for useful content online, especially from GitHub repositories and existing examples. Instead of only reading theory, I had to test the code, run it, see the error, and then slowly understand what the error meant. I also learned how to use AI as a support tool during the process. Not just to generate code directly, but more importantly to help me understand the codebase, explain the errors, and guide me on what each file is doing. I think this was useful because robotics projects can become overwhelming very quickly, especially when working with ROS2 for the first time.

Overall, this task was harder than I expected. At first, I thought the simulation would be mostly about loading a robot model and moving the joints. But after doing it, I realized that a proper simulation

needs correct structure, correct control interfaces, and correct physical properties. Even though I did not fully master everything yet, I think I gained a better understanding of how ROS2 simulation works and what steps are needed before connecting it to a real robotic arm. This experience also taught me to be more patient when reading other people's code and to debug the system layer by layer instead of trying to fix everything at once.

Delon :

Responsibility: Assembly Support and Step 13 Troubleshooting

During this robotic arm project, my main role was helping with the assembly process, especially when the team reached the parts that needed correction and rechecking. One important moment for me was when the assembly problem happened around step 13. The issue was related to the motor direction and bracket orientation. At first, the mistake made the later steps harder to continue because the arm no longer matched the expected structure. Instead of treating it as only one person's mistake, I saw it as something the team had to solve together.

When the problem became clear, I helped with disassembling and reassembling the step 13 section. This part was not only about removing screws and putting them back. We had to understand why the orientation was wrong, compare it with the manual again, and make sure the structure after correction could continue into the later steps. I think this was an important lesson because in assembly, a small change in one part can affect the whole pipeline. If one motor or bracket is placed incorrectly, the problem does not stay in that step. It carries forward and creates more work for the next steps.

I also tried to be understanding during this process. In a team project, mistakes can happen easily, especially when the manual has angle notes and the structure is not fully obvious at first. Blaming someone would not help the assembly move forward. What mattered more was slowing down, checking the evidence, and fixing the problem carefully. This taught me that teamwork is not only about doing my own assigned part. It is also about helping the team recover when something goes wrong.

Another difficulty was that our screwdrivers did not fit the space well. Some areas were too tight, and it was hard to hold the nut while tightening the screw. Because of that, we had to improvise. Willem helped by using his finger as a temporary bolt holder while I tried to fit my fingers into the small space and tighten the screw. It worked in the moment, but looking back, it also showed the limitation of our tools and planning. A better setup would have been to prepare smaller tools, tweezers, or pliers earlier so the process would be safer and more stable.

From this experience, I learned that assembly is not just following steps mechanically. It requires patience, communication, and verification. When something does not fit, we should not force it blindly. We should check whether the problem comes from the tool, the angle, the screw, or the previous step. I also learned that being a good teammate means staying calm when mistakes happen and helping turn the problem into a learning process.

Overall, my contribution was not only helping rebuild step 13, but also supporting the team when the assembly became difficult. This project showed me that careful engineering depends on both technical checking and team attitude. A team becomes stronger when members can correct mistakes together instead of just pointing them out.

Theresia :

Responsibility: Arduino, ESP32

Building a 6-axis robotic arm from scratch is a project that looks incredibly straightforward on paper but immediately tests your problem solving skills in reality. Before I even opened the Arduino IDE to write any code, me and my teammates spent quite some time on the physical assembly phase. Putting together the aluminum brackets, routing the fragile jumper wires through the articulating joints, and figuring out how to mount the servos so they wouldn't bind against each other was a massive learning curve. There is a huge difference between drawing a kinematic diagram and actually assembling it, making sure we used the right screw and making sure we connected the right parts took most of the assembly time. Once the hardware was fully assembled, tightened down, and sitting on the lab bench, another challenge was unlocked which was to get it to move using a remote control.

Initially, the control plan I was referencing relied on a rather clunky, multi-board setup. It used a standard Arduino paired with a PHPoC (PHP on Chip) shield just to host the web interface and parse network commands. As soon as I started putting it together, I realized this architecture was incredibly inefficient. Relying on secondary networking hardware felt complicated and unnecessary, so I made the decision to completely change the approach and pivot the entire system over to a single ESP32. Transitioning to the ESP32 was a game-changer, but it also meant rewriting the backend entirely in native C++. Me and my teammates spent a few days in the Arduino IDE, figuring out how to combine the original php and Arduino code so that it can properly work on the esp32. We ended up configuring the esp32 to act as its own Wi-Fi Access Point, host the raw HTML and JavaScript web server directly from memory, and manage high-speed WebSocket connections in real-time. Eliminating the need for an external router or a secondary network shield not only cleaned up the physical wiring but made the entire data pipeline significantly faster and far more robust.

Getting the backend communicating smoothly was a huge win, but translating pure mathematical intent into physical robot arm movement was a completely different obstacle. To keep the ESP32's processor free from lag and overheating, I decided to offload all the heavy Inverse Kinematics calculations to the client side. I programmed the browser's JavaScript engine to handle the math. When a user drags the target node around on the 2D web canvas, the script constantly calculates the exact joint angles required to reach that (x, y) coordinate using an expanded application of the Law of Cosines.

However, during the code review of the reference equations, I stumbled onto a critical, system-breaking bug. I noticed that the expanded formula included a division by $2\cos\alpha$. Mathematically, this creates a dangerous singularity. If the robotic arm ever attempted to reach an angle of exactly 90 degrees or 270 degrees, the cosine of that angle would be zero, triggering a divide-by-zero error that would instantly crash the entire system. Discovering that flaw was a huge relief. To fix it, I made software limits directly into the UI's JavaScript, clamping the input values so the target could never mathematically reach those exact crash-inducing coordinates. Building that fail-safe mechanism made the kinematic engine work flawlessly.

Having perfect math on paper is completely useless if it tells the physical robot to smash into its own base. The most rewarding part of this entire build was bridging that gap between mapping the theoretical 2D web interface to the harsh physical reality of a 3D robot arm. To do this, we built a custom hardware abstraction layer in C++. We created a series of calibration arrays that intercepted the "perfect" mathematical angles coming from the UI and tweaked them before they ever reached the servos. This calibration matrix was a lifesaver. It allowed me to easily reverse the PWM logic for servos that I had been forced to mount backward during the physical assembly. I could add offset degrees for joints that were resting slightly crooked due to hardware tolerances, and most importantly, I could set hard physical movement boundaries to prevent the claw from tearing out its own wiring. I mapped the UI controls to these calibrated channels. The left virtual wheel controlled the base rotation, while the vertical slider adjusted the claw's grip. Watching the physical arm mirror the web canvas movements smoothly and intuitively was an incredible feeling of validation.

As if the embedded C++ and web integration weren't enough, this project also pushed me to look beyond direct microcontroller programming and dive into the world of professional robotics simulation. While I had the physical arm working, I also got the chance to understand how this process scales in the industry. The co-learning activity introduced me to the world of ROS2 and Gazebo. Instead of risking the physical hardware on untested movement algorithms, I learned how to model the robot's description format and simulate its kinematics in Gazebo's virtual 3D environment. Visualizing the robot's environment and testing control nodes in a pure software simulation before deploying them to the physical ESP32 gave me a much deeper understanding of modern, professional robotics

pipelines. It bridged the gap between the hobbyist level of moving servos and the industrial standard of node-based robotic control.

Looking back at the entire journey of this build, I realize this project taught me a lot more about actual system design than simply following a step-by-step tutorial ever would have. It forced me to explore every engineering aspect. I was a mechanical assembler routing wires, an embedded systems engineer optimizing ESP32 performance in C++, a frontend developer building WebSocket interfaces in JavaScript, a mathematician debugging trigonometric singularities, and finally, a simulation engineer navigating ROS2 and Gazebo. It was undeniably a challenging grind, filled with frustrating moments of debugging communication protocols and recalibrating servos. Yet, walking away with a fully functional, custom-ported kinematic arm that I understand inside and out proves that the immense technical effort in this Co-learning project was entirely worth it. Moving forward, I gained a lot of new and memorable experiences, this includes the funny and stressful moments during the assembly, the pain of debugging the code and the satisfaction of seeing the successful product. Co-learning taught me how to collaborate well with people from different backgrounds whilst also expanding my technical skills in terms of assembly and programming.

Benedict :

Responsibility: Assembly

I joined this team to experience what it means to build a tangible project that is functional and integrates a lot of different systems. What we did is a 6DOF fully controllable robotic arm with grippers that can do a lot of things. Due to the extensive nature of the project, each member made their own contributions in making the robotic arm. I did some of the assembly of the arm, and frankly, it was not hard (at least my part). We were given an instruction booklet to follow in building the base, the joints and the grippers (the motors too). Although the vague and unclear nature of the booklet requires us to figure out what we are supposed to do, it teaches us how to piece several different components together and see the big idea of what it is supposed to look like and what it is supposed to do. This is, I think, a very important skill in engineering, visualizing how each component functions in a device and how the final product would look if you pieced them together, basically looking at the big picture and observing how the little things contribute to the final product.

I am not from a mechanical engineering department so I would not know about the concepts, ideas, and rules that come into effect when talking about assembly, but I did learn the various components you need in building a robotic arm. I observed in building a robot, you need copious amounts of screws and fasteners, each with its own measurements, in order to ensure and protect the stability and structure of the robotic arm. These components are small, so particular care must be kept in mind when building with them. Then you got the actuators, the moving parts; which are the servo motors. These are the ones that actually move the joints from the robot. The actual robot itself is made up of various different enclosures that give space for the servos and the wires so that it can be integrated together. The arm is mounted on a base, which allows for 360 degree rotation, and then the arm itself is controlled by an Arduino microcontroller which gives signals to control the robot.

The assembly is straightforward, sure; but integrating them all together? Personally I didn't contribute a lot in this aspect, but I can tell that this part of the process is basically an engineering headache. You need a PC to flash code to the ESP32, which relies on Wi-Fi to transmit signals from the remote control and UART signals to the microcontroller, which then uses GPIO signals to control the robotic arm. This process is not a straightforward one, as in every step it can go haywire and you won't know for what reason it occurred, so that's why I can understand the pain it comes through when building a project of this nature.

Overall, I came to this project mainly as an observer, even though I did some assembly, I was there mainly to learn the strenuous design process that comes when building this project. It was extremely worthwhile. Even though the actual sculpting was done mostly by someone else, I did experience what it means to build an engineering project outside of academics. The thing is, engineering is an iterative process, you try, fail, change your approach, and try again. This is one of the most paradoxical things about engineering, even though you may improve on your thinking and decision-making skills, it will cause you a lot of pain and regret, and this is the sacrifice one might take when taking on a project as big as this.

Jason :

Responsibility: Team Support and Assembly Checking

During this robotic arm project, my role was mainly to support the team during the assembly process, especially when the group became unsure about certain steps. I was not the main person handling the coding or leading the whole assembly, but I tried to help when the process became confusing or when the team needed to slow down and recheck something.

One thing I noticed from this project is that assembly problems can easily become worse if everyone starts rushing or guessing. Some parts looked simple in the manual, but once the brackets, screws, motors, and angles had to match together, small uncertainties became more important. When the team got stuck, I tried to help by tracing the problem back to the previous step and checking whether the structure still matched the manual. Sometimes this meant simply asking the team to pause and look again instead of immediately forcing the part to fit.

I also learned that the team atmosphere matters during hands-on work. When screws were difficult to tighten or when a section had to be redone, it was easy for people to become frustrated. I tried to keep the mood lighter when I could, not because the mistake was unimportant, but because being too tense made it harder for everyone to think clearly and communicate properly.

Overall, this project taught me that supporting a team does not always mean doing the most visible part. Sometimes it means helping the group stay patient, check the basics, and continue working

without blaming each other. My contribution was small compared with some of the technical work, but I learned that even small support can help the team move forward more smoothly.